appendix B
Django in a production environment

If you've come from a world where most of your Python gets run locally or you send your script to a friend through email, moving to the world of the web can be rather daunting. Even if you're an old pro at packaging things and submitting them to PyPI, the web has its own set of challenges. When you write something using Django, or any other web framework, you're no longer just writing code. If you want to put your project on the web, you're now dealing with infrastructure. It isn't quite as scary as it sounds, but there are some things you need to be aware of.

> **NOTE** Throughout this appendix, I will be mentioning companies that provide a variety of hosting services. This is not an endorsement; searching the web for a provider name and "alternatives" or "competitors" is a good way to get a list of choices to choose from.

Ask five different developers on the best way to put Django into production, and you'll likely get at least six opinions. Part of that has to do with scale: Is your site for you and your friends, or is it going to handle a gajillion users? How you go into production depends on this question and, of course, the associated question of the costs therein. This appendix isn't meant as an authoritative guide but more as an overview of the choices that are out there.

B.1 Parts of a web deployment

Throughout this book, you've been using the Django development server, which, as is deftly found within its name, is only for development. It isn't hardened against attack, and it isn't optimized for performance. Even if it was, a lot of the content on your web page doesn't need Django. For many of the parts of a page, the framework is overkill. Think about all the steps involved in running a view: a